

Technical Documentation DS project (Workflow)

This is a multi-vendor system in which you have multiple restaurants, and you can have a cart for each restaurant

The ability to create a user account and log into the system is present for normal users

System Admin should be able to create a restaurant and a restaurant manager account for that particular restaurant.

A restaurant manager should be able to create categories, create a menu item, and assign categories for those items

A restaurant manager should be able to create a promo code that's relevant for their restaurant, and there should be limitations, such as cart total, to determine if the promo code is valid.

Users should be able to see all the restaurants that are available within their area and choose one.

After the user has chosen the restaurant, they can add items to the cart and then proceed to payment

Before proceeding to payments, the user can enter a promo code to get a discount. Need to check if the promo code is available for the relevant restaurant.

In payments, the user can use the payment gateway to create the payment, and as a result, an order will be created.

In payment services, the payment records need to be stored in their relevant database.

This order is visible to the relevant restaurant manager's dashboard, where they can view all the available orders under them.

Once an order is complete, the request will be sent to the delivery driver. The order modal also contains the location of the user, and the location of the restaurant by default will be attached to it.

This order request goes to the delivery service, and it will assign a delivery driver based on their location or proximity to the restaurants, and also the number of orders occupied by them as in processing

The delivery driver can also see the list of orders that have been assigned to them.

They can pick up the order and give it to the customers.

The order has many processing stages

1. Food Processing
2. Delivery Processing

Throughout all the above processes, notifications need to be sent to the user and also the restaurant manager if new promotions and menu items are added.

The driver should be able to get notifications if a delivery is assigned to them.

The restaurant manager should be able to make their store available and unavailable as well. They can enable and disable items as well. They can attach discounted prices to the items as well by updating is Discounted attribute of the item

Food Delivery System: Comprehensive Technical Documentation

TABLE OF CONTENTS

1. Executive Summary
2. System Architecture
3. Microservices Breakdown
4. Database Design
5. API Design
6. Communication Patterns
7. User Workflows
8. Security Architecture
9. Deployment Strategy
10. Monitoring and Observability
11. Fault Tolerance and Resilience
12. Development Guidelines
13. Testing Strategy
14. Appendices

1. Executive Summary

This document outlines the comprehensive architecture for a distributed microservice-based food delivery system. The platform enables users to order food from multiple restaurants, with support for restaurant management, menu customization, promotions, ordering, payment processing, and delivery coordination. The system implements a modern cloud-native architecture with emphasis on scalability, resilience, and maintainability.

1.1 Business Objectives

- Create a platform connecting customers, restaurants, and delivery personnel
- Support multi-vendor restaurant management
- Enable location-based restaurant discovery
- Facilitate seamless ordering and payment processes
- Support promotional activities through discount codes

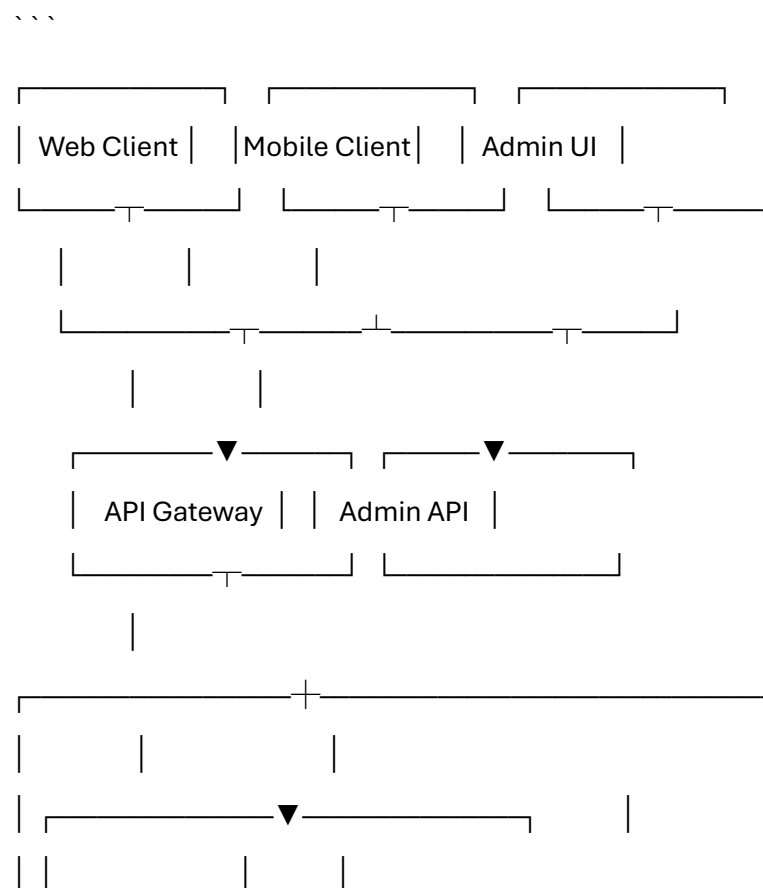
- Provide real-time order tracking and notifications
- Optimize delivery routes and driver assignments

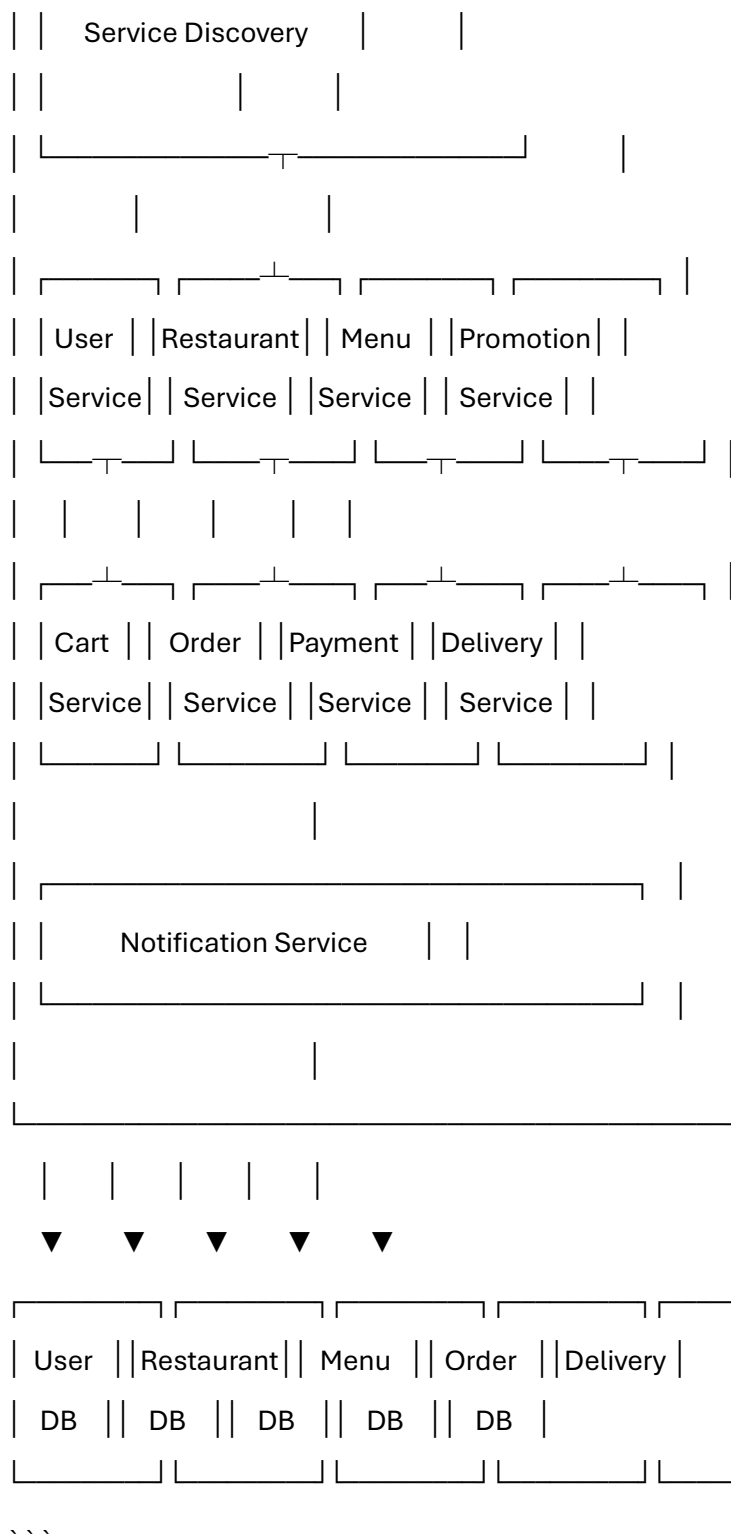
1.2 Key Technical Goals

- Implement decoupled microservices for independent scaling and maintenance
- Ensure high availability and fault tolerance
- Support real-time notifications and updates
- Provide secure payment processing
- Enable efficient location-based queries
- Implement effective caching strategies
- Support multiple client platforms (web, mobile)

2. System Architecture

2.1 High-Level Architecture Diagram





2.2 Technology Stack

Frontend

- Web: React.js with Redux

- Mobile: React Native

- Admin Dashboard: React.js with Material UI

Backend

- API Gateway: Spring Cloud Gateway
- Microservices: Spring Boot
- Service Discovery: Netflix Eureka
- Message Broker: RabbitMQ
- Caching: Redis

Data Storage

- Primary Databases: PostgreSQL (relational data)
- Document Store: MongoDB (menus, reviews, etc.)
- Caching: Redis
- Search Engine: Elasticsearch (restaurant search)

DevOps & Infrastructure

- Containerization: Docker
- Orchestration: Kubernetes
- CI/CD: Jenkins, GitHub Actions
- Monitoring: Prometheus, Grafana, ELK Stack
- Cloud Provider: AWS/Azure/GCP

3. Microservices Breakdown

3.1 API Gateway

- **Responsibility**: Entry point for all client requests
- **Key Functions**:
 - Request routing to appropriate services
 - Authentication and authorization
 - Rate limiting
 - Request/response transformation
 - API documentation (Swagger)

3.2 User Service

- **Responsibility**: Manage user accounts and profiles
- **Key Functions**:
 - User registration and authentication
 - Profile management
 - Address management
 - User preferences
 - Session management
- **Database**: User database (PostgreSQL)

3.3 Restaurant Service

- **Responsibility**: Manage restaurant information
- **Key Functions**:
 - Restaurant registration and management
 - Store availability control
 - Restaurant manager account creation
 - Location-based restaurant discovery
 - Rating and review management
- **Database**: Restaurant database (PostgreSQL)

3.4 Menu Service

- **Responsibility**: Manage restaurant menus
- **Key Functions**:
 - Menu item creation and management
 - Category management
 - Pricing and discounts
 - Item availability control
 - Menu search and filtering
- **Database**: Menu database (MongoDB)

3.5 Promotion Service

- **Responsibility**: Manage promotional offers and discounts
- **Key Functions**:
 - Promo code creation and management
 - Validation rules (cart value, restaurant, date)
 - Discount calculations
 - Usage tracking
- **Database**: Promotion database (PostgreSQL)

3.6 Cart Service

- **Responsibility**: Manage user shopping carts
- **Key Functions**:
 - Create carts per restaurant
 - Add/remove/update items
 - Calculate prices and apply promotions
 - Cart persistence
- **Database**: Cart database (Redis + PostgreSQL)

3.7 Order Service

- **Responsibility**: Process and manage orders
- **Key Functions**:
 - Order creation and tracking
 - Order status management
 - Order history
 - Processing stage transitions
- **Database**: Order database (PostgreSQL)

3.8 Payment Service

- **Responsibility**: Process payments
- **Key Functions**:
 - Payment gateway integration
 - Transaction processing
 - Payment status tracking

- Refunds management
- **Database**: Payment database (PostgreSQL)
- **External Integrations**: Payment gateways (Stripe, PayPal)

3.9 Delivery Service

- **Responsibility**: Manage delivery operations
- **Key Functions**:
 - Driver assignment algorithms
 - Location tracking
 - Delivery status management
 - ETA calculations
 - Route optimization
- **Database**: Delivery database (PostgreSQL + MongoDB for location data)
- **External Integrations**: Mapping services (Google Maps)

3.10 Notification Service

- **Responsibility**: Manage all system notifications
- **Key Functions**:
 - Push notifications
 - Email notifications
 - SMS notifications
 - In-app alerts
- **Database**: Notification database (MongoDB)
- **External Integrations**: FCM, Twilio, SendGrid

4. Database Design

4.1 User Database

Users Table

` ``

- user_id (PK)

- email
- password_hash
- first_name
- last_name
- phone_number
- user_type (CUSTOMER, RESTAURANT_MANAGER, DELIVERY_DRIVER, ADMIN)
- created_at
- updated_at
- is_active
- ...

Addresses Table

- ...
- address_id (PK)
- user_id (FK)
- address_line1
- address_line2
- city
- state
- postal_code
- country
- latitude
- longitude
- is_default
- address_type (HOME, WORK, OTHER)
- ...

UserPreferences Table

- ...
- preference_id (PK)
- user_id (FK)
- notification_preferences

- dietary_restrictions
- favorite_cuisines
- language_preference
- ...

4.2 Restaurant Database

Restaurants Table

- ...
- restaurant_id (PK)
- name
- description
- logo_url
- banner_image_url
- cuisine_type
- address_id (FK)
- latitude
- longitude
- is_active
- is_available
- average_rating
- created_at
- updated_at
- ...

RestaurantManagers Table

- ...
- manager_id (PK)
- user_id (FK)
- restaurant_id (FK)
- is_primary
- access_level

`, `

OperatingHours Table

`, `

- hours_id (PK)
- restaurant_id (FK)
- day_of_week
- opening_time
- closing_time
- is_closed_all_day

`, `

4.3 Menu Database

Categories Collection

`, `

- category_id
- restaurant_id
- name
- description
- display_order
- is_active
- created_at
- updated_at

`, `

MenuItem Collection

`, `

- item_id
- restaurant_id
- category_id
- name

- description
- price
- discounted_price
- image_url
- is_vegetarian
- is_vegan
- is_gluten_free
- allergens
- is_available
- is_featured
- is_discounted
- preparation_time
- created_at
- updated_at
- ...

4.4 Promotion Database

PromoCodes Table

- ...
- promo_id (PK)
- restaurant_id (FK)
- code
- description
- discount_type (PERCENTAGE, FIXED_AMOUNT)
- discount_value
- minimum_cart_value
- maximum_discount
- start_date
- end_date
- max_usage_count
- current_usage_count

- is_active
- created_by
- created_at
- updated_at
- ...

PromoUsage Table

- ...
- usage_id (PK)
- promo_id (FK)
- user_id (FK)
- order_id (FK)
- discount_amount
- used_at
- ...

4.5 Cart Database

Carts Table

- ...
- cart_id (PK)
- user_id (FK)
- restaurant_id (FK)
- promo_code_id (FK)
- subtotal
- discount
- tax
- delivery_fee
- total
- created_at
- updated_at
- ...

CartItems Table

...

- cart_item_id (PK)
- cart_id (FK)
- menu_item_id (FK)
- quantity
- unit_price
- special_instructions
- added_at

...

4.6 Order Database

Orders Table

...

- order_id (PK)
- user_id (FK)
- restaurant_id (FK)
- cart_id (FK)
- delivery_address_id (FK)
- payment_id (FK)
- driver_id (FK)
- order_status (CREATED, PAYMENT_PENDING, PAYMENT_COMPLETED, FOOD_PROCESSING, READY_FOR_PICKUP, DELIVERY_PROCESSING, DELIVERED, CANCELLED)
- subtotal
- discount
- tax
- delivery_fee
- total
- placed_at
- estimated_delivery_time

- actual_delivery_time
- special_instructions
- ...

OrderItems Table

- ...
- order_item_id (PK)
- order_id (FK)
- menu_item_id (FK)
- quantity
- unit_price
- special_instructions
- ...

OrderStatusHistory Table

- ...
- history_id (PK)
- order_id (FK)
- status
- changed_by
- changed_at
- notes
- ...

4.7 Payment Database

Payments Table

- ...
- payment_id (PK)
- order_id (FK)
- user_id (FK)
- payment_method (CREDIT_CARD, DEBIT_CARD, WALLET, COD)

- payment_status (PENDING, COMPLETED, FAILED, REFUNDED)
- amount
- transaction_id
- gateway_reference
- created_at
- updated_at
- ...

Refunds Table

- ...
- refund_id (PK)
- payment_id (FK)
- amount
- reason
- status
- initiated_by
- initiated_at
- completed_at
- ...

4.8 Delivery Database

Drivers Table

- ...
- driver_id (PK)
- user_id (FK)
- vehicle_type
- vehicle_number
- license_number
- current_latitude
- current_longitude
- is_active

- is_available
- current_order_count
- average_rating
- ...

DeliveryAssignments Table

- ...
- assignment_id (PK)
- order_id (FK)
- driver_id (FK)
- status (ASSIGNED, ACCEPTED, PICKED_UP, DELIVERED, CANCELLED)
- assigned_at
- accepted_at
- pickup_at
- delivered_at
- pickup_latitude
- pickup_longitude
- dropoff_latitude
- dropoff_longitude
- distance
- estimated_time
- ...

DriverLocationHistory Collection

- ...
- history_id
- driver_id
- latitude
- longitude
- timestamp
- battery_level
- speed

`, `

4.9 Notification Database

NotificationTemplates Collection

`, `

- template_id
- type (EMAIL, SMS, PUSH, IN_APP)
- event_type (ORDER_PLACED, ORDER_CONFIRMED, ORDER_READY, ORDER_PICKED_UP, ORDER_DELIVERED, etc.)
- subject
- content
- variables
- is_active

`, `

Notifications Collection

`, `

- notification_id
- user_id
- template_id
- content
- type
- status (SENT, DELIVERED, READ, FAILED)
- reference_id (order_id, promotion_id, etc.)
- reference_type
- created_at
- sent_at
- read_at

`, `

5. API Design

5.1 API Standards

- RESTful API design principles
- JSON for request/response bodies
- JWT for authentication
- Versioning via URL path (/api/v1/...)
- Standard error responses
- Pagination for collection endpoints
- Filtering and sorting capabilities

5.2 Key API Endpoints

User Service

...

- POST /api/v1/users/register - Register new user
- POST /api/v1/users/login - Authenticate user
- GET /api/v1/users/profile - Get user profile
- PUT /api/v1/users/profile - Update user profile
- GET /api/v1/users/addresses - List user addresses
- POST /api/v1/users/addresses - Add new address
- PUT /api/v1/users/addresses/{id} - Update address
- DELETE /api/v1/users/addresses/{id} - Delete address

...

Restaurant Service

...

- GET /api/v1/restaurants - List restaurants with filters
- GET /api/v1/restaurants/{id} - Get restaurant details
- POST /api/v1/restaurants - Create new restaurant (admin)
- PUT /api/v1/restaurants/{id} - Update restaurant (manager)
- PUT /api/v1/restaurants/{id}/availability - Update availability status

GET /api/v1/restaurants/nearby - Get nearby restaurants
POST /api/v1/restaurants/{id}/managers - Add restaurant manager
GET /api/v1/restaurants/{id}/reviews - Get restaurant reviews
POST /api/v1/restaurants/{id}/reviews - Add restaurant review
...

Menu Service

...
GET /api/v1/restaurants/{id}/menu - Get restaurant menu
GET /api/v1/restaurants/{id}/categories - Get menu categories
POST /api/v1/restaurants/{id}/categories - Create category (manager)
PUT /api/v1/restaurants/{id}/categories/{id} - Update category (manager)
GET /api/v1/restaurants/{id}/items - Get menu items
POST /api/v1/restaurants/{id}/items - Create menu item (manager)
PUT /api/v1/restaurants/{id}/items/{id} - Update menu item (manager)
PUT /api/v1/restaurants/{id}/items/{id}/availability - Update item availability
PUT /api/v1/restaurants/{id}/items/{id}/discount - Set item discount
...

Promotion Service

...
GET /api/v1/restaurants/{id}/promos - List promo codes
POST /api/v1/restaurants/{id}/promos - Create promo code (manager)
PUT /api/v1/restaurants/{id}/promos/{id} - Update promo code (manager)
DELETE /api/v1/restaurants/{id}/promos/{id} - Delete promo code (manager)
POST /api/v1/carts/{id}/apply-promo - Apply promo code to cart
DELETE /api/v1/carts/{id}/remove-promo - Remove promo code from cart
...

Cart Service

...
GET /api/v1/carts - Get current user's carts

GET /api/v1/carts/{id} - Get specific cart
POST /api/v1/carts - Create new cart
DELETE /api/v1/carts/{id} - Delete cart
POST /api/v1/carts/{id}/items - Add item to cart
PUT /api/v1/carts/{id}/items/{item_id} - Update cart item
DELETE /api/v1/carts/{id}/items/{item_id} - Remove item from cart
...

Order Service

...
POST /api/v1/orders - Create order from cart
GET /api/v1/orders - List user orders
GET /api/v1/orders/{id} - Get order details
PUT /api/v1/orders/{id}/status - Update order status (manager)
GET /api/v1/restaurants/{id}/orders - List restaurant orders (manager)
...

Payment Service

...
POST /api/v1/payments - Create payment for order
GET /api/v1/payments/{id} - Get payment details
POST /api/v1/payments/{id}/refund - Request refund
GET /api/v1/users/{id}/payments - List user payments
...

Delivery Service

...
GET /api/v1/drivers - List available drivers
PUT /api/v1/drivers/location - Update driver location
PUT /api/v1/drivers/availability - Update driver availability
GET /api/v1/drivers/assignments - Get driver's assignments
PUT /api/v1/deliveries/{id}/status - Update delivery status

GET /api/v1/orders/{id}/tracking - Get delivery tracking info

...

Notification Service

...

GET /api/v1/notifications - Get user notifications

PUT /api/v1/notifications/{id}/read - Mark notification as read

PUT /api/v1/users/notification-preferences - Update notification preferences

...

6. Communication Patterns

6.1 Synchronous Communication (REST)

- Used for direct user interactions requiring immediate response

- Examples:

- Restaurant search
- Menu browsing
- Cart operations
- User profile management

6.2 Asynchronous Communication (Event-Driven)

Event Types

- OrderCreatedEvent
- OrderStatusChangedEvent
- PaymentCompletedEvent
- PaymentFailedEvent
- DeliveryAssignedEvent
- DeliveryStatusChangedEvent
- MenuItemCreatedEvent
- PromoCodeCreatedEvent

Message Broker Configuration

- RabbitMQ exchanges and queues
- Dead letter queues for failed message handling
- Message persistence
- Event schema versioning

6.3 Service-to-Service Communication

Internal API Calls

- Service discovery through Eureka
- Circuit breakers with Hystrix
- Retry policies
- Timeouts

7. User Workflows

7.1 User Registration and Authentication

...

1. User submits registration form (email, password, personal details)
2. User Service validates input and creates account
3. Confirmation email sent via Notification Service
4. User confirms email
5. User can now login with credentials
6. JWT token issued upon successful authentication

...

7.2 Restaurant and Manager Setup

...

1. Admin creates restaurant record with basic details

2. System generates restaurant manager account
3. Manager receives credentials via email
4. Manager logs in and updates restaurant profile
5. Manager sets operating hours
6. Manager creates menu categories
7. Manager adds menu items with details
8. Manager sets restaurant availability
- ...

7.3 Menu and Promotion Management

...

1. Restaurant manager logs in to dashboard
2. Manager creates/edits menu categories
3. Manager adds/updates menu items
4. Items can be assigned to categories
5. Manager can set availability of items
6. Manager can set discounted prices
7. Manager creates promo codes with restrictions:
 - Minimum cart value
 - Valid date range
 - Maximum discount amount
 - Usage limits
8. System notifies users of new promotions
- ...

7.4 Order Placement

...

1. User browses restaurants by location
2. User selects restaurant and views menu
3. User adds items to cart

4. System creates and maintains cart per restaurant
5. User can modify cart (add/remove/update items)
6. User enters promo code (optional)
7. System validates promo code against restaurant and rules
8. User proceeds to checkout
9. User selects/confirms delivery address
10. User selects payment method
11. Payment Service processes payment
12. On successful payment, Order Service creates order
13. Order confirmation sent to user
14. Order appears in restaurant dashboard

...

7.5 Order Processing

...

1. Restaurant manager sees new order in dashboard
2. Manager accepts order
3. Order status changed to "Food Processing"
4. User notified of status change
5. When food is ready, manager updates status to "Ready for Pickup"
6. Delivery Service assigns driver based on:
 - Proximity to restaurant
 - Current driver workload
 - Driver availability
7. Selected driver receives notification
8. Driver accepts delivery assignment
9. Driver arrives at restaurant and updates status to "Picked Up"
10. Order status changes to "Delivery Processing"
11. User can track driver location in real-time
12. Driver delivers order and marks as "Delivered"
13. User receives delivery confirmation

14. User can rate order and delivery experience

...

8. Security Architecture

8.1 Authentication

- JWT-based authentication
- Token expiration and refresh mechanism
- OAuth 2.0 for social logins
- Multi-factor authentication for sensitive operations
- Secure password storage with bcrypt

8.2 Authorization

- Role-based access control (RBAC)
 - Customer
 - Restaurant Manager
 - Delivery Driver
 - System Admin
- Fine-grained permissions
- Resource-level access control

8.3 Data Protection

- Encryption at rest
- Encryption in transit (TLS/SSL)
- PII data handling according to GDPR
- Payment information security (PCI DSS compliance)
- Data masking for sensitive information

8.4 API Security

- Rate limiting
- CORS configuration
- Input validation
- Prevention of common attacks:
 - SQL Injection
 - XSS
 - CSRF
 - Request forgery

8.5 Security Monitoring

- Failed login attempts tracking
- Suspicious activity detection
- Regular security audits
- Vulnerability scanning

9. Deployment Strategy

9.1 Environment Setup

- Development
- Testing/QA
- Staging
- Production

9.2 Infrastructure as Code

- Terraform for cloud resources
- Kubernetes manifests for container orchestration
- Helm charts for application deployment

9.3 Containerization

- Docker images for each service
- Multi-stage builds to minimize image size
- Container security scanning

9.4 Kubernetes Deployment

- Deployment configurations
- Service definitions
- Ingress controllers
- ConfigMaps and Secrets
- Horizontal Pod Autoscalers
- Network Policies

9.5 CI/CD Pipeline

- Source control integration (GitHub)
- Automated builds
- Unit and integration testing
- Security scanning
- Artifact generation and storage
- Blue/green deployments
- Canary releases
- Automated rollbacks

10. Monitoring and Observability

10.1 Logging Strategy

- Centralized logging with ELK Stack
- Structured log formats (JSON)

- Log levels (INFO, WARN, ERROR, DEBUG)
- Correlation IDs for request tracking
- Log rotation and retention policies

10.2 Metrics Collection

- Prometheus for metrics collection
- Service-level metrics
 - Request rate
 - Error rate
 - Response time
- Business metrics
 - Orders per hour
 - Average order value
 - Delivery time
- Infrastructure metrics
 - CPU/Memory usage
 - Network throughput
 - Disk I/O

10.3 Alerting

- Grafana alerting
- PagerDuty integration
- Alert thresholds
- Escalation policies
- On-call rotations

10.4 Distributed Tracing

- Jaeger for distributed tracing
- OpenTelemetry instrumentation

- Trace sampling
- Performance bottleneck identification

10.5 Dashboards

- Operational dashboards
- Business metrics dashboards
- Custom dashboards per team/role
- Real-time monitoring views

11. Fault Tolerance and Resilience

11.1 Circuit Breakers

- Hystrix/Resilience4j configuration
- Fallback mechanisms
- Bulkhead pattern implementation

11.2 Retry Policies

- Exponential backoff
- Jitter
- Maximum retry attempts

11.3 Rate Limiting

- API rate limiting
- Client-specific limits
- Rate limit headers

11.4 Caching Strategy

- Redis caching
- Cache invalidation strategies
- TTL configuration
- Cache-aside pattern implementation

11.5 Database Resilience

- Connection pooling
- Read replicas
- Failover mechanisms
- Data replication

11.6 Disaster Recovery

- Backup strategies
- Recovery procedures
- Recovery Point Objective (RPO)
- Recovery Time Objective (RTO)

12. Development Guidelines

12.1 Coding Standards

- Java/Kotlin coding standards
- Code formatting
- Naming conventions
- Documentation requirements

12.2 API Design Guidelines

- URI naming conventions
- HTTP method usage

- Status code usage
- Error response format
- Versioning strategy

12.3 Git Workflow

- Branching strategy (GitFlow)
- Commit message format
- Pull request process
- Code review guidelines

12.4 Documentation

- API documentation (Swagger/OpenAPI)
- README files
- Architecture Decision Records (ADRs)
- Wiki maintenance

13. Testing Strategy

13.1 Unit Testing

- Testing frameworks (JUnit, Mockito)
- Coverage requirements
- Test naming conventions
- Test data management

13.2 Integration Testing

- Service-to-service integration tests
- Database integration tests
- Test containers

13.3 API Testing

- Automated API tests
- Postman collections
- Contract testing (Spring Cloud Contract)

13.4 Performance Testing

- Load testing
- Stress testing
- Endurance testing
- JMeter test plans

13.5 Security Testing

- SAST (Static Application Security Testing)
- DAST (Dynamic Application Security Testing)
- Dependency vulnerability scanning
- Penetration testing

14. Appendices

14.1 API Schemas

- OpenAPI specifications
- JSON Schema definitions
- Example requests/responses

14.2 Database Migration Scripts

- Flyway migration scripts

- Database initialization scripts

14.3 Deployment Scripts

- Kubernetes manifests
- Helm charts
- Terraform configurations

14.4 Glossary

- Business terms
- Technical terms
- Acronyms and abbreviations

14.5 References

- External documentation links
- Related projects
- Standards and specifications

This comprehensive documentation provides a detailed blueprint for implementing the distributed food delivery system using microservice architecture. Each section contains the necessary information for development, deployment, and maintenance of the system, ensuring alignment with business requirements and technical best practices.

Similar code found with 4 license types